

SuperMa POS

Systeme de Vente et Gestion de Stock

Documentation Technique Complete

Version 2.0 | 30 mai 2026

PHP 8.3 | PostgreSQL | Vanilla JS SPA | PWA

Table des matieres

1. Introduction
2. Technologies utilisees
3. Architecture applicative
4. Structure du projet
5. Base de donnees et schema
6. API Reference (18 endpoints)
7. Interface utilisateur (SPA)
8. Authentification et securite
9. Deploiement
10. Configuration
11. Generation de documents
12. PWA et cache
13. Guide de developpement

1. Introduction

SuperMa POS est un systeme de point de vente (POS) complet, cloud-native, concu pour les commerces de detail. Il gere les ventes, le stock, les clients, les fournisseurs, les commandes fournisseurs et les utilisateurs avec un controle d'accès granulaire (RBAC).

L'application fonctionne comme une Single Page Application (SPA) cote frontend, avec une API RESTful backend en PHP et une base de donnees PostgreSQL. Elle est entierement autonome : aucun framework PHP ni bibliotheque JavaScript externe n'est utilise. La generation de documents (PDF, DOCX, XLSX) est faite sur mesure sans dependances tierces.

Fonctionnalites principales

- Point de vente avec grille produits, scan de code-barres, panier, remises
- Paiements en especes, carte bancaire (Stripe) et lien de paiement mobile
- Gestion de stock avec seuils minimum/maximum et ajustements
- Gestion des achats / commandes fournisseurs
- Tableau de bord avec indicateurs CA, transactions, top produits
- Multi-magasins avec devise et fuseau horaire par magasin
- Controle d'accès par roles (admin, manager, caissier, stock keeper)
- Export XLSX, generation DOCX et PDF (bon de commande, facture)
- PWA installable, service worker, cache offline
- Logs applicatifs structures avec niveaux et canaux

2. Technologies utilisees

Backend

Composant	Technologie	Version
Langage	PHP	8.3
Base de donnees	PostgreSQL	16+ (avec pg_trgm)
Driver BD	PDO pgsql	-
Auth	Session PHP + Bearer Token	-
Hachage	bcrypt (cost 12)	-
Conteneurisation	Docker	Alpine
Serveur web	Nginx	1.26+
Cache	Fichier (TTL 15s)	-

Frontend

Composant	Technologie
Architecture	Vanilla JS SPA (3042 lignes)
API Client	Fetch API avec Bearer token (439 lignes)
Style	CSS personnalisée (1085 lignes, variables CSS)
PWA	Service Worker + Manifest
Police	System stack (Segoe UI, system-ui)

Services externes

Service	Usage
Stripe	Traitement des paiements (API REST, sans SDK)
Railway.app	Hebergement cible principal (Docker)

3. Architecture applicative

L'application suit une architecture client-serveur classique :

- Frontend : SPA en JavaScript vanilla. Toutes les vues sont rendues cote client dans /public/js/app.js. La navigation se fait via un routeur interne (fonction navigate()).
- Backend : API RESTful en PHP. Chaque fichier dans /api/ correspond a une ressource (products, sales, stock, etc.). Le routage est centralise dans index.php (front controller).
- Base de donnees : PostgreSQL. Toutes les donnees sont isolees par store_id (multi-tenant). L'extension pg_trgm avec index GIN permet des recherches ILIKE efficaces.

Flux d'une requete

```
1. Navigateur -> index.html -> app.js (SPA shell)
2. app.js -> API (fetch avec Bearer token)
3. index.php -> routing -> api/products.php
4. products.php -> PDO -> PostgreSQL
5. Reponse JSON -> app.js -> rendu DOM
```

Multi-tenant par magasin

Chaque requete API filtre les donnees par store_id de l'utilisateur connecte. Certaines tables (suppliers, categories) autorisent les enregistrements sans store_id (store_id IS NULL) comme valeurs partagees.

4. Structure du projet

```
SuperMa/
  api/           # 17 endpoints API REST
  config/        # Configuration BD (PDO)
  deploy/        # Docker, nginx, seed, migration
  includes/      # Auth, helpers, loggers, generateurs
  logs/          # Fichiers de log applicatifs
  public/
    css/app.css  # Styles complets
    js/
      app.js     # SPA (toutes les vues)
      api.js     # Client API
      sw.js      # Service worker
      index.html # Shell SPA
      manifest.json # PWA manifest
  index.php      # Front controller
  Dockerfile
  railway.json
```

Dossier api/

Endpoint	Ressource	Methodes
/api/login	Authentification	POST, GET, DELETE
/api/products	Produits	GET, POST, PUT, DELETE
/api/categories	Categories	GET, POST, PUT, DELETE
/api/customers	Clients	GET, POST, PUT, DELETE
/api/suppliers	Fournisseurs	GET, POST, PUT, DELETE
/api/users	Utilisateurs	GET, POST, PUT, DELETE
/api/stores	Magasins	GET, POST, PUT, DELETE
/api/dashboard	Tableau de bord	GET
/api/sales	Ventes	GET, POST, DELETE
/api/roles	Roles	GET
/api/stock	Stock	GET, POST
/api/orders	Commandes	GET, POST, PUT, DELETE
/api/export	Export XLSX	GET
/api/stripe	Paiement Stripe	GET, POST
/api/logs	Logs	GET, POST
/api/sse	SSE temps reel	GET
/api/ping	Mutation signal	GET
/health	Healthcheck	GET

Dossier includes/

Fichier	Role
auth.php	Session + Bearer token, requireAuth(), getUserRoles()
bootstrap.php	Amorçage : session, erreurs, CORS, content-type JSON
cors.php	En-tetes CORS dynamiques
helpers.php	jsonResponse(), validate(), sanitize()
logger.php	Logger structure (fichiers, niveaux, canaux)
cache.php	Cache fichier avec TTL
stripe.php	API Stripe maison (cURL via stream_context_create)
pdf.php	Generateur PDF natif (format 1.4)
docx.php	Generateur DOCX (ZIP + XML OOXML)
xlsx.php	Generateur XLSX (ZIP + SpreadsheetML)

5. Base de donnees et schema

Tables principales

Table	Role	Cle primaire
users	Utilisateurs du systeme	UUID
stores	Magasins / points de vente	UUID
roles	Definitions de roles	ID serie
permissions	Permissions granulaires	ID serie
user_roles	Liaison user <-> role	composite
role_permissions	Liaison role <-> permission	composite
products	Catalogue produits	UUID
categories	Categories hierarchiques	UUID
suppliers	Fournisseurs	UUID
customers	Clients	UUID
sales	Ventes	UUID
sale_items	Lignes de vente	ID serie
payment_methods	Modes de paiement	ID serie
purchase_orders	Commandes fournisseur	UUID
purchase_order_items	Lignes de commande	ID serie
stock_movements	Mouvements de stock	ID serie

Index de performance (pg_trgm)

Le fichier deploy/migration.sql active pg_trgm et cree des index GIN sur :

- products (name, barcode, sku)
- customers (first_name, last_name, email, phone)
- suppliers (company_name, contact_name)
- sales (sale_number)
- purchase_orders (order_number)
- Index B-tree sur les foreign keys et colonnes de filtrage (store_id, status, is_active, dates).

6. API Reference

Authentication

Toutes les routes API (sauf /api/login et /health) necessitent un Bearer token envoye dans le header Authorization. Le token (64 octets hexadecimaux) est stocke dans sessionStorage cote client et dans \$_SESSION cote serveur.

La session est securisee par :

- Session start avec cookie HttpOnly, SameSite=Strict, Secure
- Regeneration d'ID apres login (session_regenerate_id(true))
- session_write_close() apres requireAuth() pour eviter le verrouillage

Format des reponses

```
// Succes
{ "success": true, "data": { ... }, "pagination": { "page": 1, "per_page": 20, "total": 150 } }

// Erreur
{ "success": false, "error": "Message d'erreur", "code": 400 }
```

Endpoints principaux

/api/products

GET : Liste paginee des produits avec filtres (search, category_id, page). Retourne aussi le nom de la categorie et du fournisseur associes.

POST : Cree un produit. Champs requis : name, selling_price. Champs optionnels : barcode, sku, category_id, supplier_id, purchase_price, tax_rate, stock_quantity, etc.

DELETE : Desactive un produit (soft delete, is_active = 0).

/api/sales

POST : Cree une vente en transaction. Verifie le stock, insere la vente et les lignes, enregistre le mouvement de stock. Champs requis : items (tableau de { product_id, quantity, unit_price }), paid_amount, payment_method_id.

DELETE : Annule une vente. Restaure le stock, enregistre un mouvement de type return.

/api/orders

POST : Cree un bon de commande fournisseur. Champs requis : supplier_id, items (tableau de { product_id, ordered_qty, unit_price }).

PUT : Met a jour le statut (ex: received -> met a jour stock_qty et received_qty pour chaque ligne).

/api/stock

POST : Ajustement de stock (transactionnel). Champs requis : product_id, quantity, movement_type. Types autorises : adjustment, inventory, loss, transfer_in, transfer_out.

/api/dashboard

GET : Retourne les statistiques du jour et du mois en cours : revenu, transactions, panier moyen, top 10 produits, ventes 14 derniers jours, ventes recentes. Cache fichier 15s.

7. Interface utilisateur (SPA)

Pages

Route	Page	Roles autorises
/dashboard	Tableau de bord	Tous (admin: stats completes)
/products	Produits	Tous
/categories	Categories	Tous
/customers	Clients	caissier, manager
/suppliers	Fournisseurs	manager, stock_keeper
/stock	Stock	manager, stock_keeper
/orders	Commandes	manager, stock_keeper
/sales	Ventes	manager, caissier
/users	Utilisateurs	admin
/stores	Magasins	admin
/logs	Journaux	admin

Composants cles

- Barre de navigation laterale avec icones et filtrage par role
- Recherche cote client pour chaque page (filtre instantane sans debounce)
 - Interface de vente (POS) : grille de produits, scanner code-barres, panier, calcul des totaux, remises
 - Dashboard : cartes KPI, graphique barres 14 jours, top produits (chart CSS uniquement)
 - Tableaux pagines avec tri et export XLSX
 - Modals pour creation / edition / consultation (formulaires dynamiques)

Gestion des vues par role

Les vues disponibles (lien de navigation) sont filtrees selon les permissions de l'utilisateur. Le tableau de bord affiche des statistiques completes uniquement pour les administrateurs ; les caissiers, gestionnaires de stock et managers voient simplement le logo du magasin et leur nom.

8. Authentification et securite

Mecanismes

- Session PHP pour le backend (HttpOnly, SameSite=Strict, Secure)
- Bearer token 64-byte hex pour les requetes AJAX (stocke sessionStorage)
- bcrypt cost 12 pour les mots de passe
- session_regenerate_id(true) apres login
- session_write_close() apres requireAuth() pour eviter le verrouillage concurrent

Permissions (RBAC)

Le systeme utilise 4 roles predefinis avec des permissions granulaires :

Role	Permissions cles
admin	Toutes les permissions
manager	Ventes, stock, commandes, clients, fournisseurs, dashboard
cashier	Ventes (creer/consulter), clients, produits, categories
stock_keeper	Stock, produits, categories, fournisseurs, commandes

Protections reseau

- Nginx bloque les acces aux dossiers /config, /includes, /logs, /deploy
- Fichiers caches (.env, .git) bloques par Nginx et .htaccess
- Headers de securite : X-Frame-Options, X-Content-Type-Options, CSP, Referrer-Policy
- CORS limite aux origines autorisees (variable CORS_ORIGIN)
- Healthcheck sans authentication (endpoint /health)

Validation des entrees

- Toutes les entrees sont validees via helpers.php (validate(), sanitize())
- Requetes parametrees PDO (prepared statements)
- CSRF : token inclus dans les requetes mutation

9. Deploiement

Docker (deploiement principal)

Le Dockerfile construit une image PHP 8.3 FPM Alpine avec Nginx compile depuis les sources :

```
# Build
docker build -t superma-pos .
# Run
docker run -p 8080:8080 --env-file .env superma-pos
```

Railway.app

Le fichier railway.json configure le deploiement :

```
{
  "build": "DOCKERFILE",
  "healthcheckPath": "/health",
  "healthcheckTimeout": 30,
  "restartPolicyMaxRetries": 5
}
```

Variables requises : DATABASE_URL (PostgreSQL), APP_ENV=production, JWT_SECRET, APP_URL.

Hebergement alternatif

- Render.com : deploiement PHP natif, 750h/mois gratuits
- Koyeb : deploiement Docker, 1 service web + 1 PostgreSQL gratuit
- Serveur dedie : PHP 8.3 + Nginx + PostgreSQL

Entrypoint (deploy/entrypoint.sh)

```
#!/bin/sh
sed -i "s/listen 8080/listen $PORT/g" /etc/nginx/http.d/default.conf
php-fpm -D
nginx -g 'daemon off;'
```

10. Configuration

Variables d'environnement

Variable	Description	Default
DB_DRIVER	Driver base de donnees	pgsql
DB_HOST	Hote base de donnees	127.0.0.1
DB_PORT	Port	5432
DB_NAME	Nom de la base	superma_pos
DB_USER	Utilisateur	superma_user
DB_PASSWORD	Mot de passe	
DATABASE_URL	URL complete (ecrase les DB_*)	
APP_NAME	Nom application	SuperMa POS
APP_ENV	Environnement	production
APP_DEBUG	Mode debug (true/false)	false
APP_URL	URL de base	
JWT_SECRET	Cle de session	
SESSION_LIFETIME	Duree session (s)	7200
CORS_ORIGIN	Origine CORS autorisee	*
STRIPE_SECRET_KEY	Cle secrete Stripe	
STRIPE_PUBLISHABLE_KEY	Cle publiable Stripe	
PORT	Port HTTP (Docker)	8080

Configuration Nginx

Deux fichiers de configuration Nginx sont fournis :

- `deploy/nginx.conf` : Configuration production avec SSL Let's Encrypt, headers securite, gzip, cache statique 7 jours, blocage des dossiers sensibles.
- `deploy/nginx-docker.conf` : Configuration Docker simplifiee (port 8080, proxy PHP-FPM).

11. Generation de documents

PDF (includes/pdf.php)

Generateur de PDF natif pour les bons de commande. Construit le format PDF 1.4 manuellement (sans bibliotheque). Gere les en-tetes, pieds de page, tableau des articles et totaux.

DOCX (includes/docx.php)

Generateur de documents Office Open XML. Cree les fichiers DOCX (ZIP contenant du XML OOXML) pour les bons de commande et les factures. Supporte les styles, tableaux, images.

XLSX (includes/xlsx.php)

Generateur de feuilles de calcul Excel. Utilise le format SpreadsheetML (ZIP + XML). Genere des exports pour toutes les ressources : produits, clients, fournisseurs, utilisateurs, magasins, categories, ventes.

12. PWA et cache

Service Worker (public/sw.js)

Le service worker utilise une strategie cache-first pour les ressources statiques (JS, CSS, images, polices) et network-first pour les appels API. Le cache est nomme superma-v2 (incremente pour forcer le re-cache apres mise a jour).

Manifest (public/manifest.json)

Le manifest PWA permet d'installer l'application sur l'ecran d'accueil avec :

- Nom : SuperMa POS
- Theme : #2563eb (bleu)
- Background : #ffffff
- Display : standalone (plein ecran sans barre navigateur)
- Icons : 192x192 et 512x512

13. Guide de developpement

Environnement local

```
# Cloner le depot
git clone <url> superma-pos && cd superma-pos

# Configurer la base de donnees
cp .env.example .env
# Editer .env avec vos identifiants PostgreSQL

# Lancer le serveur de developpement PHP
php -S localhost:8000 -t public router.php

# Initialiser la base
# Executer les fichiers deploy/seed.sql sur votre base PostgreSQL
```

Conventions de code

- PHP : declare(strict_types=1), typage fort, match expression, snake_case
- JS : IIFE global, camelCase, indentation 2 espaces
- API : Reponses JSON uniformes { success, data/error }, codes HTTP RESTful
- BD : UUIDs pour les ID, timestamps created_at/updated_at, soft delete via is_active

Ajouter une page

1. Ajouter le lien dans la fonction renderNav() (app.js)
2. Creer la fonction de rendu (render[Nom]View)
3. Ajouter la route dans navigate()
4. Si besoin : creer un endpoint dans api/

Ajouter un endpoint API

1. Creer api/maresource.php
2. Inclure bootstrap.php
3. Verifier l'auth : Auth::requireAuth()
4. Gerer les methodes HTTP via match (\$_SERVER['REQUEST_METHOD'])
5. Repondre avec jsonResponse()

Index

API	6	Navigation	7
Architecture	3	Nginx	9
Authentification	8	Pages	7
Base de donnees	5	PDF	11
Cache	12	Permissions	8
Commandes	6	PostgreSQL	5
Configuration	10	Produits	6
CSS	4	PWA	12
Dashboard	7	Railway	9
Deploiement	9	Roles	7
DOCX	11	Routes API	6
Docker	9	Securite	8
Environnement	10	Service Worker	12
Export XLSX	11	Session	8
Fonctionnalites	1	SPA	7
Fournisseurs	6	Stock	6
Frontend	7	Stripe	2
Generation docs	11	Structure projet	4
JS (app.js)	4	Technologies	2
Logs	6	Ventes	6
Multi-tenant	3	XLSX	11

Notes

Cette documentation est generee automatiquement a partir de l'analyse du code source.

Document genere le 30/05/2026

SuperMa POS v2.0

